

```

    fprintf(stderr, "Incorrect invocation. Possibilities
are:\n");
    fprintf(stderr, "\tkls /' Picks up %s as the default
partition_file *\n\n",
        argv[0], SIMULA_DEFAULT_FILE);
    fprintf(stderr, "\tkls [ partition_file ]\n", argv[0]);
    return 1;
}
if (argc == 2)
{
    sfs_file = argv[1];
}
sfs_handle = open(sfs_file, O_RDONLY);
if (sfs_handle == -1)
{
    fprintf(stderr, "Unable to browse SFS over %s\n", sfs_
file);
    return 2;
}
read(sfs_handle, &sb, sizeof(sfs_super_block_t));
if (sb.type != SIMULA_FS_TYPE)
{
    fprintf(stderr, "Invalid SFS detected. Giving up.\n");
    close(sfs_handle);
    return 3;
}
browse_sfs(sfs_handle);
close(sfs_handle);
return 0;
}

```



Note: The sample code from this article is available at http://linuxforu.com/article_source_code/june12/device-part19.zip. Download and experiment with it.

The above (shell like) program primarily reads the super block from the partition file (sfsf by default, or the file provided from command line), and then gets into browsing the file system based on that information, using the `browse_sfs()` function. Note the check performed for the valid file system on the partition file using the magic number `SIMULA_FS_TYPE`.

`browse_sfs()` prints the file system information and provides four basic file system functionalities using the following commands:

- `quit` – to quit the file system browser
- `list` – to list the current files in the file system (using `sfs_list()`)
- `create <filename>` – to create a new file in the file system (using `sfs_create(filename)`)
- `remove <filename>` – to remove an existing file from the file system (using `sfs_remove(filename)`)

Figure 1 shows the browser in action, using the above commands.

`sfs_list()` traverses through all the file entries in the partition

```

[anil@anishi Code]$ ./browse_sfs
Welcome to SFS Browsing Shell v1.0

Block size : 512 bytes
Partition size: 1024 blocks
File entry size: 64 bytes
Entry tbl size: 102 blocks
Entry count : 816

$> ?
Supported commands:
$> ? quit list create remove
$> list
$> create cool
$> list
cool 0 bytes rwx Wed May 2 19:00:48 2012
$> create hot
$> list
cool 0 bytes rwx Wed May 2 19:00:48 2012
hot 0 bytes rwx Wed May 2 19:00:58 2012
$> remove cool
$> list
hot 0 bytes rwx Wed May 2 19:00:58 2012
$> h
Unknown/Incorrect command: h
Supported commands:
$> ? quit list create <file> remove <file>
$>

```

Figure 1: Simula file system browser output

and prints all the non-null filename entries – with file name, size, permissions, and its creation time stamp. `sfs_create()` looks up for an available (null filename) entry and then updates it with the given filename, size of 0 bytes, permissions of 'rwx', and the current time stamp. And `sfs_remove()` looks up for an existing file entry with the filename to be removed, and then nullifies it. The other parts in the above code are more of basic error handling cases like invalid commands in `browse_sfs()`, existing file name in `sfs_create()`, non-existing file name in `sfs_remove()`, etc.

Note that the above application is the first-cut of a full-fledged application. So the files created right now come with just the basic fixed parameters and there is no way to change their permissions, content, etc, yet. However, adding these features is just a matter of adding commands and their corresponding functions; Shweta will provide these commands for a few more interesting features, once she works out the details of the browsing application.

Summing up

Once done with the other features as well, the next set of steps would take the project partners closer to their final goals. Watch out for who takes charge of the next step of moving the file system logic to the kernel space. **END**

By: Anil Kumar Pugalia

The author is a freelance trainer in Linux internals, Linux device drivers, embedded Linux and related topics. Prior to this, he was at Intel and Nvidia. He has been working with Linux since 1994. A gold medalist from the Indian Institute of Science, Linux and knowledge sharing are two of his many passions. Creating and playing with open source hardware is one of his hobbies. Learn more about him at <http://profession.sarika-pugs.com>, including information about his open source hardware freak-outs and his various Linux related training sessions and workshops. He can be reached at email@sarika-pugs.com.